Contents

1 Basics 1
- 1.1 .vimrc 1
- 1.2 Default 1
- 1.3 IO-optimize 1
- 1.4 PBDS 1
- 1.5 Random 1
- 1.6 run.sh 1

2 Data-Structure 1
- 2.1 2D-BIT 1
- 2.2 2D-Segment-Tree 2
- 2.3 BIT 2
- 2.4 LiChaoST.cpp 2
- 2.5 Segment-Tree 3
- 2.6 Treap 3
- 2.7 ZKW-Segment-Tree 3

3 Flow 3
- 3.1 Bipartite-Matching 3
- 3.2 Dinic 4
- 3.3 KM 4
- 3.4 MCMF 5

4 Geometry 5
- 4.1 Basic$_2D$ 5
- 4.2 BronKerbosch 6
- 4.3 Convex-Hull 7
- 4.4 Dynamic-Convex-Hull 7
- 4.5 Segmentation-Intersection 7

5 Graph 8
- 5.1 2-SAT 8
- 5.2 AP-Bridge 8
- 5.3 Biconnected-Component 8
- 5.4 Bridge-connected-Component 8
- 5.5 Close-Vertices 8
- 5.6 Disjoint-Set 9
- 5.7 Heavy-Light-Decomposition 9
- 5.8 KSP 10
- 5.9 LCA 10
- 5.10 SCC(Kosaraju) 10
- 5.11 SCC(Tarjan) 11
- 5.12 Tree-Centroid 11

6 Math 11
- 6.1 Big-Int 11
- 6.2 Extgcd 12
- 6.3 Karatsuba 12
- 6.4 Linear-Sieve 12
- 6.5 Matrix 13
- 6.6 Miller-Rabin 13
- 6.7 NTT 13
- 6.8 Pollard-Rho 13
- 6.9 Theorem 14

7 Others 14
- 7.1 Mo 14
- 7.2 Partial-Ordering 14

8 String 14
- 8.1 AC 14
- 8.2 Hash 15
- 8.3 KMP 15
- 8.4 Manacher 15
- 8.5 SA 15
- 8.6 SAIS 15
- 8.7 Z 16

1 Basics

1.1 .vimrc

```
"This file should be placed at ~/.vimrc"
se nu ai hls et ru ic is sc cul
se re=1 ts=4 sts=2 sw=4 ls=2 mouse=a
syntax on
set bg=dark
inoremap {<CR> {<CR>}<Esc>ko<tab>
```

1.2 Default

```
#include <bits/stdc++.h>
using namespace std;
void _debug() {cerr << '\n';}
template <typename A, typename... B>
void _debug(A a, B... b) {cerr << a << ' ', _debug(b...)
    ;}
#define debug(args...) cerr << '(' << (#args) << ") : ",
     _debug(args)
using ll = long long;
const int N = 2e5 + 5, inf = 1e9;

int main() {
  ios::sync_with_stdio(false), cin.tie(nullptr);
}
```

1.3 IO-optimize

```
bool rit(auto& x) {
  x = 0; char c = cin.rdbuf()->sbumpc(); bool neg = 0;
  while (!isdigit(c)) {
    if (c == EOF) return 0;
    if (c == '-') neg = 1;
    c = cin.rdbuf()->sbumpc();
  }
  while (isdigit(c))
    x = x * 10 + c - '0', c = cin.rdbuf()->sbumpc();
  return x = neg ? -x : x, 1;
}
void wit(auto x) {
  if (x < 0) cout.rdbuf()->sputc('-'), x = -x;
  char s[20], len = 0;
  do s[len++] = x % 10 + '0'; while (x /= 10);
  while (len) cout.rdbuf()->sputc(s[--len]);
}
```

1.4 PBDS

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
// #include <bits/extc++.h>
#include <bits/stdc++.h>
using namespace __gnu_pbds;
using namespace std;
template <typename T>
using rbtree = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
// less<T> :  increasing, greater<T> : decreasing
// rb_tree_tag, splay_tree_tag, ov_tree_tag

int main() {
  int x;
  rbtree<int> t, rhs, rhs2;
  t.insert(x);
  t.erase(x); // return 1 or 0
  cout << t.order_of_key(x) << '\n'; // rank
  cout << *t.find_by_order(x) << '\n'; // x-th
  cout << *t.lower_bound(x) << '\n'; // iterator >= x
  cout << *t.upper_bound(x) << '\n'; // iterator > x
  t.join(rhs); // merge // same type, no duplicate
      elements
  t.split(x, rhs2); // tree : elements <= x, rhs :
      elements > x
}
```

1.5 Random

```
#include <random>
#include <chrono>
#include <algorithm>
mt19937 rng(chrono::system_clock::now().time_since_epoch
    ().count());
int randint(int lb, int ub) {
  return uniform_int_distribution<int>(lb, ub)(rng);
}
// shuffle(v.begin(), v.end(), rng);
```

1.6 run.sh

```
test -e ./$1.out && rm ./$1.out
g++ -std=c++17 -O2 -Wall -Wextra -fsanitize=undefined -
    fsanitize=address -o $1.out $1.cpp
```

```
echo "finish"
test -e ./$1.out && ./$1.out
```

2 Data-Structure

2.1 2D-BIT

```
using ll = long long;
const int N = 1000 + 5;
ll bit[N][N];
int n, m, a[N][N];
void build() {
  for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
      bit[i][j] = a[i][j];
  for (int b = 1; b << 1 <= max(n, m); b <<= 1) {
    for (int i = b; i + b <= n; i += b << 1)
      for (int j = 1; j <= m; j++)
        bit[i + b][j] += bit[i][j];
    for (int i = 1; i <= n; i++)
      for (int j = b; j + b <= m; j += b << 1)
        bit[i][j + b] += bit[i][j];
  }
}
void upd(int x, int y, int v) {
  for (int i = x; i <= n; i += i & -i)
    for (int j = y; j <= m; j += j & -j)
      bit[i][j] += v;
}
ll qry(int x, int y) {
  ll ret = 0;
  for (int i = x; i; i -= i & -i)
    for (int j = y; j; j -= j & -j)
      ret += bit[i][j];
  return ret;
}
ll qry(int x1, int y1, int x2, int y2) { // closed-
    interval
  return qry(x2, y2) - qry(x1 - 1, y2) - qry(x2, y1 - 1)
      + qry(x1 - 1, y1 - 1);
}
```

2.2 2D-Segment-Tree

```
const int inf = 1e9;
#define lc(x) (x << 1)
#define rc(x) (x << 1 | 1)
int N, M; // N : row max, M : col max
struct seg {
  vector<int> st;
  void pull(int);
  void merge(const seg&, const seg&, int, int, int);
  void build(int, int, int);
  void upd(int, int, int, int, int);
  int qry(int, int, int, int, int);
  seg(int size): st(size << 2 | 1) {}
};
void seg::pull(int id) {
  st[id] = max(st[lc(id)], st[rc(id)]);
}
void seg::merge(const seg& a, const seg& b, int id = 1,
    int l = 1, int r = M) {
  st[id] = max(a.st[id], b.st[id]);
  if (l == r) return;
  int m = (l + r) >> 1;
  merge(a, b, lc(id), l, m), merge(a, b, rc(id), m + 1,
      r);
}
void seg::build(int id = 1, int l = 1, int r = M) {
  if (l == r) {cin >> st[id]; return;}
  int m = (l + r) >> 1;
  build(lc(id), l, m), build(rc(id), m + 1, r);
  pull(id);
}
void seg::upd(int x, int v, int id = 1, int l = 1, int r
    = M) {
  if (l == r) {st[id] = v; return;}
  int m = (l + r) >> 1;
  if (x <= m) upd(x, v, lc(id), l, m);
  else upd(x, v, rc(id), m + 1, r);
  pull(id);
}
int seg::qry(int ql, int qr, int id = 1, int l = 1, int
    r = M) {
  if (ql <= l && r <= qr) return st[id];
  int m = (l + r) >> 1, ret = -inf;
  if (ql <= m) ret = max(ret, qry(ql, qr, lc(id), l, m))
      ;
  if (qr > m) ret = max(ret, qry(ql, qr, rc(id), m + 1,
      r));
  return ret;
}

struct segseg {
  vector<seg> st;
  void pull(int, int);
  void build(int, int, int);
  void upd(int, int, int, int, int, int);
  int qry(int, int, int, int, int, int, int);
  segseg(int n, int m): st(n << 2 | 1, seg(m)) {}
};
void segseg::pull(int id, int x) {
  st[id].upd(x, max(st[lc(id)].qry(x, x), st[rc(id)].qry
      (x, x)));
}
void segseg::build(int id = 1, int l = 1, int r = N) {
  if (l == r) {st[id].build(); return;}
  int m = (l + r) >> 1;
  build(lc(id), l, m), build(rc(id), m + 1, r);
  st[id].merge(st[lc(id)], st[rc(id)]);
}
void segseg::upd(int y, int x, int v, int id = 1, int l
    = 1, int r = N) {
  if (l == r) {st[id].upd(x, v); return;}
  int m = (l + r) >> 1;
  if (y <= m) upd(y, x, v, lc(id), l, m);
  else upd(y, x, v, rc(id), m + 1, r);
  pull(id, x);
}
int segseg::qry(int y1, int y2, int x1, int x2, int id =
    1, int l = 1, int r = N) {
  if (y1 <= l && r <= y2) return st[id].qry(x1, x2);
  int m = (l + r) >> 1, ret = -inf;
  if (y1 <= m) ret = max(ret, qry(y1, y2, x1, x2, lc(id)
      , l, m));
  if (y2 > m) ret = max(ret, qry(y1, y2, x1, x2, rc(id),
      m + 1, r));
  return ret;
}
```

2.3 BIT

```
#define lowbit(x) (x & -x)
const int N = 2e5 + 5;
int n; // maxn
ll a[N], bit1[N], bit2[N];
ll sum(ll* bit, int x) {
  ll ret = 0;
  for (; x; x -= lowbit(x)) ret += bit[x];
  return ret;
}
void upd(ll* bit, int x, ll v) {
  for (; x <= n; x += lowbit(x)) bit[x] += v;
}
ll qry(int x) {
  return (x + 1) * sum(bit1, x) - sum(bit2, x);
}
ll qry(int l, int r) { // [l, r]
  return qry(r) - qry(l - 1);
}
void upd(int l, int r, ll v) { // [l, r]
  upd(bit1, l, v), upd(bit2, l, l * v);
  upd(bit1, r + 1, -v), upd(bit2, r + 1, (r + 1) * -v);
}
void build(ll* a, int n) { // O(n)
  for (int i = 1; i <= n; i++) bit1[i] = a[i] - a[i -
      lowbit(i)];
  for (int i = n; i; i--) a[i] -= a[i - 1];
  for (int i = 1; i <= n; i++) a[i] = a[i - 1] + a[i] *
      i;
  for (int i = 1; i <= n; i++) bit2[i] = a[i] - a[i -
      lowbit(i)];
}
```

2.4 LiChaoST.cpp

```
struct LiChao_min {
  struct line {
    LL m, c;
    line(LL _m = 0, LL _c = 0) {
      m = _m;
      c = _c;
    }
    LL eval(LL x) { return m * x + c; }
  };
  struct node {
    node *l, *r;
    line f;
    node(line v) {
      f = v;
      l = r = NULL;
    }
  };
  typedef node *pnode;
  pnode root;
  int sz;
#define mid ((l + r) >> 1)
  void insert(line &v, int l, int r, pnode &nd) {
    if (!nd) {
      nd = new node(v);
      return;
    }
    LL trl = nd->f.eval(l), trr = nd->f.eval(r);
    LL vl = v.eval(l), vr = v.eval(r);
    if (trl <= vl && trr <= vr) return;
    if (trl > vl && trr > vr) {
      nd->f = v;
      return;
    }
    if (trl > vl) swap(nd->f, v);
    if (nd->f.eval(mid) < v.eval(mid))
      insert(v, mid + 1, r, nd->r);
    else swap(nd->f, v), insert(v, l, mid, nd->l);
  }
  LL query(int x, int l, int r, pnode &nd) {
    if (!nd) return LLONG_MAX;
    if (l == r) return nd->f.eval(x);
    if (mid >= x)
      return min(
        nd->f.eval(x), query(x, l, mid, nd->l));
    return min(
      nd->f.eval(x), query(x, mid + 1, r, nd->r));
  }
  /* -sz <= query_x <= sz */
  void init(int _sz) {
    sz = _sz + 1;
    root = NULL;
  }
  void add_line(LL m, LL c) {
    line v(m, c);
    insert(v, -sz, sz, root);
  }
  LL query(LL x) { return query(x, -sz, sz, root); }
};
```

2.5 Segment-Tree

```
#define lc(x) (x << 1)
#define rc(x) (x << 1 | 1)
const int N = 2e5 + 5;
int seg[N << 2];
void build(int id, int l, int r) { // [1, n]
  if (l == r) return cin >> seg[id], void();
  int m = (l + r) >> 1;
  build(lc(id), l, m), build(rc(id), m + 1, r);
  seg[id] = max(seg[lc(id)], seg[rc(id)]);
}
void upd(int id, int l, int r, int x, int v) {
  if (l == r) return seg[id] = v, void();
  int m = (l + r) >> 1;
  if (x <= m) upd(lc(id), l, m, x, v);
  else upd(rc(id), m + 1, r, x, v);
  seg[id] = max(seg[lc(id)], seg[rc(id)]);
}
int qry(int id, int l, int r, int ql, int qr) { // [ql,
    qr]
  if (ql <= l && r <= qr) return seg[id];
  if (ql > r || qr < l) return 0;
  int m = (l + r) >> 1;
  return max(qry(lc(id), l, m, ql, qr), qry(rc(id), m +
      1, r, ql, qr));
}
```

2.6 Treap

```
#include <bits/stdc++.h>
using namespace std;
mt19937 rng;
struct node {
  node *l, *r;
  int v, p, s; bool t; // val, pri, size, tag
  void pull();
  void push();
  node(int vv = 0): v(vv), p(rng()), s(1), t(0) {l = r =
      nullptr;}
};
int sz(node* o) {return o ? o->s : 0;}
void node::pull() {s = (l ? l->s : 0) + (r ? r->s : 0) +
    1;}
void node::push() {
  if (t) {
    swap(l, r);
    if (l) l->t ^= 1;
    if (r) r->t ^= 1;
    t = 0;
  }
}
node* merge(node* a, node* b) {
  if (!a || !b) return a ? : b;
  if (a->p < b->p) return a->push(), a->r = merge(a->r,
      b), a->pull(), a;
  else return b->push(), b->l = merge(a, b->l), b->pull
      (), b;
}
void split(node* o, node*& a, node*& b, int k) { // a <
    k, b >= k
  if (!o) return a = b = nullptr, void();
  o->push();
  if (o->v < k) a = o, split(o->r, a->r, b, k);
  else b = o, split(o->l, a, b->l, k);
  o->pull();
}
void insert(node*& o, int k) {
  node *a, *b;
  split(o, a, b, k), o = merge(a, merge(new node(k), b))
      ;
}
void ssplit(node* o, node*& a, node*& b, int k) { //
    split first k things
  if (!o) return a = b = nullptr, void();
  if (sz(o->l) + 1 <= k) a = o, ssplit(o->r, a->r, b, k
      - sz(o->l) - 1);
  else b = o, ssplit(o->l, a, b->l, k);
  o->pull();
}
void reverse(node* o, int l, int r) {
  node *a, *b, *c;
  ssplit(o, a, b, l - 1), ssplit(b, b, c, r - l + 1);
  b->t ^= 1, o = merge(a, merge(b, c));
}
```

2.7 ZKW-Segment-Tree

```
#define lc(x) (x << 1)
#define rc(x) (x << 1 | 1)
constexpr int N = 2e5 + 1;
int seg[N << 1], n;
void upd(int x, int v) {
  for (seg[x += n] = v; x > 1; x >>= 1)
    seg[x >> 1] = max(seg[x], seg[x ^ 1]);
}
int qry(int l, int r) { // [ql, qr)
  int ret = -1e9;
  for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
    if (l & 1) ret = max(ret, seg[l++]);
    if (r & 1) ret = max(ret, seg[--r]);
  }
  return ret;
```

```
}
void build() { // [0, n)
  for (int i = 0; i < n; i++)
    cin >> seg[i + n];
  for (int i = n - 1; i; i--)
    seg[i] = max(seg[lc(i)], seg[rc(i)]);
}
```

3 Flow

3.1 Bipartite-Matching

```
struct Bipartite_Matching { // 0-base
  int l, r;
  int mp[MAXN], mq[MAXN];
  int dis[MAXN], cur[MAXN];
  vector<int> G[MAXN];
  bool dfs(int u) {
    for (int &i = cur[u]; i < SZ(G[u]); ++i) {
      int e = G[u][i];
      if (!~mq[e] || (dis[mq[e]] == dis[u] + 1 && dfs(mq
          [e])))
        return mp[mq[e] = u] = e, 1;
    }
    dis[u] = -1;
    return 0;
  }
  bool bfs() {
    int rt = 0;
    queue<int> q;
    fill_n(dis, l, -1);
    for (int i = 0; i < l; ++i)
      if (!~mp[i])
        q.push(i), dis[i] = 0;
    while (!q.empty()) {
      int u = q.front();
      q.pop();
      for (int e : G[u])
        if (!~mq[e])
          rt = 1;
        else if (!~dis[mq[e]]) {
          q.push(mq[e]);
          dis[mq[e]] = dis[u] + 1;
        }
    }
    return rt;
  }
  int matching() {
    int rt = 0;
    fill_n(mp, l, -1);
    fill_n(mq, r, -1);
    while (bfs()) {
      fill_n(cur, l, 0);
      for (int i = 0; i < l; ++i)
        if (!~mp[i] && dfs(i))
          ++rt;
    }
    return rt;
  }
  void add_edge(int s, int t) {
    G[s].pb(t);
  }
  void init(int _l, int _r) {
    l = _l, r = _r;
    for (int i = 0; i < l; ++i)
      G[i].clear();
  }
};
```

3.2 Dinic

```
struct MaxFlow { // 0-base
  struct edge {
    int to, cap, flow, rev;
  };
  vector<edge> G[MAXN];
  int s, t, dis[MAXN], cur[MAXN], n;
  int dfs(int u, int cap) {
    if (u == t || !cap) return cap;
    for (int &i = cur[u]; i < (int)G[u].size(); ++i) {
      edge &e = G[u][i];
      if (dis[e.to] == dis[u] + 1 && e.flow != e.cap) {
        int df = dfs(e.to, min(e.cap - e.flow, cap));
        if (df) {
          e.flow += df;
          G[e.to][e.rev].flow -= df;
          return df;
        }
      }
    }
    dis[u] = -1;
    return 0;
  }
  bool bfs() {
    fill_n(dis, n, -1);
    queue<int> q;
    q.push(s), dis[s] = 0;
    while (!q.empty()) {
      int tmp = q.front();
      q.pop();
      for (auto &u : G[tmp])
        if (!~dis[u.to] && u.flow != u.cap) {
          q.push(u.to);
          dis[u.to] = dis[tmp] + 1;
        }
    }
    return dis[t] != -1;
  }
  int maxflow(int _s, int _t) {
    s = _s, t = _t;
    int flow = 0, df;
    while (bfs()) {
      fill_n(cur, n, 0);
      while ((df = dfs(s, INF))) flow += df;
    }
    return flow;
  }
  void init(int _n) {
    n = _n;
    for (int i = 0; i < n; ++i) G[i].clear();
  }
  void reset() {
    for (int i = 0; i < n; ++i)
      for (auto &j : G[i]) j.flow = 0;
  }
  void add_edge(int u, int v, int cap) {
    G[u].pb(edge{v, cap, 0, (int)G[v].size()});
    G[v].pb(edge{u, 0, 0, (int)G[u].size() - 1});
  }
};
```

3.3 KM

```
struct KM { // 0-base
  int w[MAXN][MAXN], hl[MAXN], hr[MAXN], slk[MAXN], n;
  int fl[MAXN], fr[MAXN], pre[MAXN], qu[MAXN], ql, qr;
  bool vl[MAXN], vr[MAXN];
  void init(int _n) {
    n = _n;
    for (int i = 0; i < n; ++i)
      for (int j = 0; j < n; ++j) w[i][j] = -INF;
  }
  void add_edge(int a, int b, int wei) {
    w[a][b] = wei;
  }
  bool Check(int x) {
    if (vl[x] = 1, ~fl[x])
      return vr[qu[qr++] = fl[x]] = 1;
    while (~x) swap(x, fr[fl[x] = pre[x]]);
    return 0;
  }
  void Bfs(int s) {
    fill(slk, slk + n, INF);
    fill(vl, vl + n, 0), fill(vr, vr + n, 0);
    ql = qr = 0, qu[qr++] = s, vr[s] = 1;
    while (1) {
      int d;
      while (ql < qr)
        for (int x = 0, y = qu[ql++]; x < n; ++x)
          if (!vl[x] &&
              slk[x] >= (d = hl[x] + hr[y] - w[x][y]))
            if (pre[x] = y, d) slk[x] = d;
            else if (!Check(x)) return;
      d = INF;
```

```
      for (int x = 0; x < n; ++x)
        if (!vl[x] && d > slk[x]) d = slk[x];
      for (int x = 0; x < n; ++x) {
        if (vl[x]) hl[x] += d;
        else slk[x] -= d;
        if (vr[x]) hr[x] -= d;
      }
      for (int x = 0; x < n; ++x)
        if (!vl[x] && !slk[x] && !Check(x)) return;
    }
  }
  int Solve() {
    fill(fl, fl + n, -1), fill(fr, fr + n, -1),
         fill(hr, hr + n, 0);
    for (int i = 0; i < n; ++i)
      hl[i] = *max_element(w[i], w[i] + n);
    for (int i = 0; i < n; ++i) Bfs(i);
    int res = 0;
    for (int i = 0; i < n; ++i) res += w[i][fl[i]];
    return res;
  }
};
```

3.4 MCMF

```
struct MinCostMaxFlow { // 0-base
  struct Edge {
    ll from, to, cap, flow, cost, rev;
  } *past[N];
  vector<Edge> G[N];
  int inq[N], n, s, t;
  ll dis[N], up[N], pot[N];
  bool BellmanFord() {
    fill_n(dis, n, INF), fill_n(inq, n, 0);
    queue<int> q;
    auto relax = [&](int u, ll d, ll cap, Edge *e) {
      if (cap > 0 && dis[u] > d) {
        dis[u] = d, up[u] = cap, past[u] = e;
        if (!inq[u]) inq[u] = 1, q.push(u);
      }
    };
    relax(s, 0, INF, 0);
    while (!q.empty()) {
      int u = q.front();
      q.pop(), inq[u] = 0;
      for (auto &e : G[u]) {
        ll d2 = dis[u] + e.cost + pot[u] - pot[e.to];
        relax(e.to, d2, min(up[u], e.cap - e.flow), &e);
      }
    }
    return dis[t] != INF;
  }
  void solve(int _s, int _t, ll &flow, ll &cost, bool
      neg = true) {
    s = _s, t = _t, flow = 0, cost = 0;
    if (neg) BellmanFord(), copy_n(dis, n, pot);
    for (; BellmanFord(); copy_n(dis, n, pot)) {
      for (int i = 0; i < n; ++i) dis[i] += pot[i] - pot
          [s];
      flow += up[t], cost += up[t] * dis[t];
      for (int i = t; past[i]; i = past[i]->from) {
        auto &e = *past[i];
        e.flow += up[t], G[e.to][e.rev].flow -= up[t];
      }
    }
  }
  void init(int _n) {
    n = _n, fill_n(pot, n, 0);
    for (int i = 0; i < n; ++i) G[i].clear();
  }
  void add_edge(ll a, ll b, ll cap, ll cost) {
    G[a].pb(Edge{a, b, cap, 0, cost, SZ(G[b])});
    G[b].pb(Edge{b, a, 0, 0, -cost, SZ(G[a]) - 1});
  }
};
```

4 Geometry

4.1 Basic$_2D$

```
// Courtesy of Jinkela
const double PI = atan2(0.0, -1.0);
template<typename T>
struct point {
  T x, y;
  point() {}
  point(const T&x, const T&y): x(x), y(y) {}
  point operator+(const point &b)const {
    return point(x + b.x, y + b.y);
  }
  point operator-(const point &b)const {
    return point(x - b.x, y - b.y);
  }
  point operator*(const T &b)const {
    return point(x * b, y * b);
  }
  point operator/(const T &b)const {
    return point(x / b, y / b);
  }
  bool operator==(const point &b)const {
    return x == b.x && y == b.y;
  }
  T dot(const point &b)const { return x * b.x + y * b.y;
      }
  T cross(const point &b)const { return x * b.y - y * b.
      x; }
  point normal()const { //求法向量
    return point(-y, x);
  }
  T abs2()const { return dot(*this); }
  T rad(const point &b)const { //兩向量的弧度
    return fabs(atan2(fabs(cross(b)), dot(b)));
  }
  T getA()const { //對x軸的弧度
    T A = atan2(y, x); //超過180度會變負的
    if (A <= -PI / 2)A += PI * 2;
    return A;
  }
};
template<typename T>
struct line {
  line() {}
  point<T> p1, p2;
  T a, b, c; //ax+by+c=0
  line(const point<T>&x, const point<T>&y): p1(x), p2(y)
      {}
  void pton() { //轉成一般式
    a = p1.y - p2.y; b = p2.x - p1.x; c = -a * p1.x - b
        * p1.y;
  }
  T ori(const point<T> &p)const { //點和有向直F的關F,
      >0左邊、=0在F上<0右邊
    return (p2 - p1).cross(p - p1);
  }
  T btw(const point<T> &p)const { //點投影落在F段上<=0
    return (p1 - p).dot(p2 - p);
  }
  bool point_on_segment(const point<T>&p)const {
    return ori(p) == 0 && btw(p) <= 0;
  }
  T dis2(const point<T> &p, bool is_segment = 0)const {
      //點跟直F/F段的距離平方
    point<T> v = p2 - p1, v1 = p - p1;
    if (is_segment) {
      point<T> v2 = p - p2;
      if (v.dot(v1) <= 0)return v1.abs2();
      if (v.dot(v2) >= 0)return v2.abs2();
    }
    T tmp = v.cross(v1); return tmp * tmp / v.abs2();
  }
  T seg_dis2(const line<T> &l)const { //兩F段距離平方
    return min({dis2(l.p1, 1), dis2(l.p2, 1), l.dis2(p1,
        1), l.dis2(p2, 1)});
  }
  point<T> projection(const point<T> &p)const { //點對直
      F的投影
    point<T> n = (p2 - p1).normal();
    return p - n * (p - p1).dot(n) / n.abs2();
  }
  point<T> mirror(const point<T> &p)const {
    //點對直F的鏡射，要先呼叫pton轉成一般式
    point<T> R; T d = a * a + b * b;
    R.x = (b * b * p.x - a * a * p.x - 2 * a * b * p.y -
        2 * a * c) / d;
```

```
    R.y = (a * a * p.y - b * b * p.y - 2 * a * b * p.x -
         2 * b * c) / d;
    return R;
  }
  bool parallel(const line &l)const {
    return (p1 - p2).cross(l.p1 - l.p2) == 0;
  }
};
template<typename T>
struct polygon {
  polygon() {}
  vector<point<T> > p;//逆時針順序
  T double_signed_area()const {
    T ans = 0;
    for (int i = p.size() - 1, j = 0; j < (int)p.size();
         i = j++)
      ans += p[i].cross(p[j]);
    return ans;
  }
  point<T> center_of_mass()const {
    T cx = 0, cy = 0, w = 0;
    for (int i = p.size() - 1, j = 0; j < (int)p.size();
         i = j++) {
      T a = p[i].cross(p[j]);
      cx += (p[i].x + p[j].x) * a; cy += (p[i].y + p[j].
          y) * a;
      w += a;
    } return point<T>(cx / 3 / w, cy / 3 / w);
  }
  int ahas(const point<T>& t)const { //點是否在簡單多邊
      形F, 是的話回傳1、在邊上回傳-1、否則回傳0
    int c = 0; //Works for clockwise input as well
    for (int i = 0, j = p.size() - 1; i < p.size(); j =
        i++) {
      if (line<T>(p[i], p[j]).point_on_segment(t))return
          -1;
      if ((p[i].y > t.y) != (p[j].y > t.y)) {
        T L = (t.x - p[i].x) * (p[j].y - p[i].y);
        T R = (p[j].x - p[i].x) * (t.y - p[i].y);
        if (p[j].y < p[i].y) {L = -L; R = -R;}
        if (L < R)c = !c;
      }
    } return c;
  }
  int point_in_convex(const point<T>&x)const {
    int l = 1, r = (int)p.size() - 2;
    while (l <= r) { //點是否在凸多邊形F, 是的話回傳1、
        在邊上回傳-1、否則回傳0
      int mid = (l + r) / 2;
      T a1 = (p[mid] - p[0]).cross(x - p[0]);
      T a2 = (p[mid + 1] - p[0]).cross(x - p[0]);
      if (a1 >= 0 && a2 <= 0) {
        T res = (p[mid + 1] - p[mid]).cross(x - p[mid]);
        return res > 0 ? 1 : (res >= 0 ? -1 : 0);
      }
      if (a1 < 0)r = mid - 1; else l = mid + 1;
    } return 0;
  }
  vector<T> getA()const { //凸包邊對x軸的夾角
    vector<T>res;//一定是遞增的
    for (size_t i = 0; i < p.size(); ++i)
      res.push_back((p[(i + 1) % p.size()] - p[i]).getA
          ());
    return res;
  }
  bool line_intersect(const vector<T>&A, const line<T> &
      l)const { //O(logN)
    int f1 = upper_bound(A.begin(), A.end(), (l.p1 - l.
        p2).getA()) - A.begin();
    int f2 = upper_bound(A.begin(), A.end(), (l.p2 - l.
        p1).getA()) - A.begin();
    return l.cross_seg(line<T>(p[f1], p[f2]));
  }
  T diam() {
    int n = p.size(), t = 1;
    T ans = 0; p.push_back(p[0]);
    for (int i = 0; i < n; i++) {
      point<T> now = p[i + 1] - p[i];
      while (now.cross(p[t + 1] - p[i]) > now.cross(p[t]
          - p[i]))t = (t + 1) % n;
      ans = max(ans, (p[i] - p[t]).abs2());
    } return p.pop_back(), ans;
  }
  T min_cover_rectangle() {
    int n = p.size(), t = 1, r = 1, l;
    if (n < 3)return 0; //也可以做最小周長矩形
    T ans = 1e99; p.push_back(p[0]);
    for (int i = 0; i < n; i++) {
      point<T> now = p[i + 1] - p[i];
      while (now.cross(p[t + 1] - p[i]) > now.cross(p[t]
          - p[i]))t = (t + 1) % n;
      while (now.dot(p[r + 1] - p[i]) > now.dot(p[r] - p
          [i]))r = (r + 1) % n;
      if (!i)l = r;
      while (now.dot(p[l + 1] - p[i]) <= now.dot(p[l] -
          p[i]))l = (l + 1) % n;
      T d = now.abs2();
      T tmp = now.cross(p[t] - p[i]) * (now.dot(p[r] - p
          [i]) - now.dot(p[l] - p[i])) / d;
      ans = min(ans, tmp);
    } return p.pop_back(), ans;
  }
  T dis2(polygon &pl) { //凸包最近距離平方
    vector<point<T> > &P = p, &Q = pl.p;
    int n = P.size(), m = Q.size(), l = 0, r = 0;
    for (int i = 0; i < n; ++i)if (P[i].y < P[l].y)l = i
        ;
    for (int i = 0; i < m; ++i)if (Q[i].y < Q[r].y)r = i
        ;
    P.push_back(P[0]), Q.push_back(Q[0]);
    T ans = 1e99;
    for (int i = 0; i < n; ++i) {
      while ((P[l] - P[l + 1]).cross(Q[r + 1] - Q[r]) <
          0)r = (r + 1) % m;
      ans = min(ans, line<T>(P[l], P[l + 1]).seg_dis2(
          line<T>(Q[r], Q[r + 1])));
      l = (l + 1) % n;
    } return P.pop_back(), Q.pop_back(), ans;
  }
  static int sign(const point<T>&t) {
    return (t.y ? t.y : t.x) < 0;
  }
  static bool angle_cmp(const line<T>& A, const line<T>&
      B) {
    point<T> a = A.p2 - A.p1, b = B.p2 - B.p1;
    return sign(a) < sign(b) || (sign(a) == sign(b) && a
        .cross(b) > 0);
  }
  int halfplane_intersection(vector<line<T> > &s) {
    sort(s.begin(), s.end(), angle_cmp); //F段左側F該
        F段半平面
    int L, R, n = s.size();
    vector<point<T> > px(n);
    vector<line<T> > q(n);
    q[L = R = 0] = s[0];
    for (int i = 1; i < n; ++i) {
      while (L < R && s[i].ori(px[R - 1]) <= 0)--R;
      while (L < R && s[i].ori(px[L]) <= 0)++L;
      q[++R] = s[i];
      if (q[R].parallel(q[R - 1]) && q[--R].ori(s[i].p1)
          > 0)q[R] = s[i];
      if (L < R)px[R - 1] = q[R - 1].line_intersection(q
          [R]);
    }
    while (L < R && q[L].ori(px[R - 1]) <= 0)--R;
    p.clear();
    if (R - L <= 1)return 0;
    px[R] = q[R].line_intersection(q[L]);
    for (int i = L; i <= R; ++i)p.push_back(px[i]);
    return R - L + 1;
  }
};
```

4.2 BronKerbosch

```
// Courtesy of Jinkela
struct maximalCliques {
  using Set = vector<int>;
  size_t n; //1-base
  vector<Set> G;
  static Set setUnion(const Set &A, const Set &B) {
    Set C(A.size() + B.size());
    auto it = set_union(A.begin(), A.end(), B.begin(), B
        .end(), C.begin());
```

```
    C.erase(it, C.end());
    return C;
  }
  static Set setIntersection(const Set &A, const Set &B)
      {
    Set C(min(A.size(), B.size()));
    auto it = set_intersection(A.begin(), A.end(), B.
        begin(), B.end(), C.begin());
    C.erase(it, C.end());
    return C;
  }
  static Set setDifference(const Set &A, const Set &B) {
    Set C(min(A.size(), B.size()));
    auto it = set_difference(A.begin(), A.end(), B.begin
        (), B.end(), C.begin());
    C.erase(it, C.end());
    return C;
  }
  void BronKerbosch1(Set R, Set P, Set X) {
    if (P.empty() && X.empty()) {
      // R form an maximal clique
      return;
    }
    for (auto v : P) {
      BronKerbosch1(setUnion(R, {v}), setIntersection(P,
          G[v]), setIntersection(X, G[v]));
      P = setDifference(P, {v});
      X = setUnion(X, {v});
    }
  }
  void init(int _n) {
    G.clear();
    G.resize((n = _n) + 1);
  }
  void addEdge(int u, int v) {
    G[u].emplace_back(v);
    G[v].emplace_back(u);
  }
  void solve(int n) {
    Set P;
    for (int i = 1; i <= n; ++i) {
      sort(G[i].begin(), G[i].end());
      G[i].erase(unique(G[i].begin(), G[i].end()), G[i].
          end());
      P.emplace_back(i);
    }
    BronKerbosch1({}, P, {});
  }
};
```

4.3 Convex-Hull

```
#define f first
#define s second
#define ALL(x) (x).begin(), (x).end()
template <typename T>
pair<T, T> operator-(const pair<T, T>& a, const pair<T,
    T>& b) {
  return {a.f - b.f, a.s - b.s};
}
template <typename T>
int cross(const pair<T, T>& o, const pair<T, T>& a,
    const pair<T, T>& b) {
  auto p = a - o, q = b - o;
  return p.f * q.s - q.f * p.s;
}
template <typename T>
vector<pair<T, T>> convex_hull(vector<pair<T, T>> hull)
    {
  if (hull.size() <= 2) return hull;
  sort(ALL(hull));
  vector<pair<T, T>> stk;
  int n = hull.size();
  for (int i = 0; i < n; i++) {
    while (stk.size() >= 2 && cross(stk.end()[-2], stk.
        end()[-1], hull[i]) <= 0)
      stk.pop_back();
    stk.push_back(hull[i]);
  }
  for (int i = n - 2, t = stk.size() + 1; i >= 0; i--) {
    while ((int)stk.size() >= t && cross(stk.end()[-2],
        stk.end()[-1], hull[i]) <= 0)
      stk.pop_back();
    stk.push_back(hull[i]);
  }
  return stk.pop_back(), stk;
}
```

4.4 Dynamic-Convex-Hull

```
struct Line {
  ll a, b, l = MIN, r = MAX;
  Line(ll a, ll b): a(a), b(b) {}
  ll operator()(ll x) const {
    return a * x + b;
  }
  bool operator<(Line b) const {
    return a < b.a;
  }
  bool operator<(ll b) const {
    return r < b;
  }
};

ll iceil(ll a, ll b) {
  if (b < 0) a *= -1, b *= -1;
  if (a > 0) return (a + b - 1) / b;
  else return a / b;
}

ll intersect(Line a, Line b) {
  return iceil(a.b - b.b, b.a - a.a);
}

struct DynamicConvexHull {
  multiset<Line, less<>> ch;

  void add(Line ln) {
    auto it = ch.lower_bound(ln);
    while (it != ch.end()) {
      Line tl = *it;
      if (tl(tl.r) <= ln(tl.r)) {
        it = ch.erase(it);
      }
      else break;
    }
    auto it2 = ch.lower_bound(ln);
    while (it2 != ch.begin()) {
      Line tl = *prev(it2);
      if (tl(tl.l) <= ln(tl.l)) {
        it2 = ch.erase(prev(it2));
      }
      else break;
    }
    it = ch.lower_bound(ln);
    if (it != ch.end()) {
      Line tl = *it;
      if (tl(tl.l) >= ln(tl.l)) ln.r = tl.l - 1;
      else {
        ll pos = intersect(ln, tl);
        tl.l = pos;
        ln.r = pos - 1;
        ch.erase(it);
        ch.insert(tl);
      }
    }
    it2 = ch.lower_bound(ln);
    if (it2 != ch.begin()) {
      Line tl = *prev(it2);
      if (tl(tl.r) >= ln(tl.r)) ln.l = tl.r + 1;
      else {
        ll pos = intersect(tl, ln);
        tl.r = pos - 1;
        ln.l = pos;
        ch.erase(prev(it2));
        ch.insert(tl);
      }
    }
    if (ln.l <= ln.r) ch.insert(ln);
  }

  ll query(ll pos) {
    auto it = ch.lower_bound(pos);
    if (it == ch.end()) return 0;
```

```
    return (*it)(pos);
  }
};
```

4.5 Segmentation-Intersection

```
int sign(ll x) {
    return (x > 0 ? 1 : (x < 0 ? -1 : 0));
}

ll cross(pair<ll, ll> o, pair<ll, ll> a, pair<ll, ll> b)
     {
    return (a.first - o.first) * (b.second - o.second) -
         (a.second - o.second) * (b.first - o.first);
}

bool intersect1D(ll a, ll b, ll c, ll d) {
    if (a > b) swap(a, b);
    if (c > d) swap(c, d);
    return max(a, c) <= min(b, d);
}

bool intersect2D(pair<ll, ll> a, pair<ll, ll> b, pair<ll
    , ll> c, pair<ll, ll> d) {
    return intersect1D(a.first, b.first, c.first, d.
        first)
           && intersect1D(a.second, b.second, c.second,
               d.second)
           && sign(cross(a, b, c)) * sign(cross(a, b, d)
               ) <= 0
           && sign(cross(c, d, a)) * sign(cross(c, d, b)
               ) <= 0;
}
```

5 Graph

5.1 2-SAT

```
struct TwoSAT {
  int n;
  Scc scc;
  void init(int _n) {
    // (0,1),(2,3),...
    n = _n; scc.init(n * 2);
  }
  void add_disjunction(int a, int na, int b, int nb) {
    a = 2 * a ^ na, b = 2 * b ^ nb;
    scc.addEdge(a ^ 1, b);
    scc.addEdge(b ^ 1, a);
  }
  vector<int> solve() {
    scc.solve();
    vector<int> assignment(n, 0);
    for (int i = 0; i < n; i++) {
      if (scc.bln[2 * i] == scc.bln[2 * i ^ 1]) return
          {};
      assignment[i] = scc.bln[2 * i] > scc.bln[2 * i ^
          1];
    }
    return assignment;
  }
};
```

5.2 AP-Bridge

```
#define eb emplace_back
using pii = pair<int, int>;
const int N = 2e5 + 5;

int d[N], low[N];
vector<int> g[N];
vector<int> ap;
vector<pii> bridge;

void dfs(int x, int p) {
  d[x] = ~p ? d[p] + 1 : 1, low[x] = d[x];
  int cnt = 0;
  bool isap = 0;
  for (const auto& i : g[x]) {
    if (i == p) continue;
    if (!d[i]) {
      dfs(i, x), cnt++;
      if (d[x] <= low[i]) isap = 1;
      if (d[x] < low[i]) bridge.eb(x, i);
      low[x] = min(low[x], low[i]);
    }
    low[x] = min(low[x], d[i]);
  }
  if (p == -1 && cnt < 2) isap = 0;
  if (isap) ap.eb(x);
}
```

5.3 Biconnected-Component

```
// beware of multiple inputs
#define ep emplace
#define eb emplace_back
const int N = 2e5 + 5;

int d[N], low[N];
vector<int> g[N];
vector<vector<int>> bcc;
stack<int> st;

void dfs(int x, int p) {
  d[x] = ~p ? d[p] + 1 : 1, low[x] = d[x];
  st.ep(x);
  for (const auto& i : g[x]) {
    if (i == p) continue;
    if (!d[i]) {
      dfs(i, x);
      low[x] = min(low[x], low[i]);
      if (d[x] <= low[i]) {
        int tmp;
        bcc.eb();
        do tmp = st.top(), st.pop(), bcc.back().eb(tmp);
             while (tmp != x);
        st.ep(x);
      }
    }
    low[x] = min(low[x], d[i]);
  }
}
```

5.4 Bridge-connected-Component

```
#define ep emplace
constexpr int N = 2e5 + 1;

int d[N], low[N], bcc[N], nbcc;
vector<int> g[N];
stack<int> st;

void dfs(int x, int p) {
  d[x] = ~p ? d[p] + 1 : 1, low[x] = d[x];
  st.ep(x);
  for (const auto& i : g[x]) {
    if (i == p) continue;
    if (!d[i]) {
      dfs(i, x);
      low[x] = min(low[x], low[i]);
    }
    low[x] = min(low[x], d[i]);
  }
  if (low[x] == d[x]) {
    nbcc++;
    int tmp;
    do tmp = st.top(), st.pop(), bcc[tmp] = nbcc; while
        (tmp != x);
  }
}
```

5.5 Close-Vertices

```
#include <iostream>
#include <vector>
#include <bitset>
#include <algorithm>
#include <cstring>
using namespace std;
int l, w;
vector<pair<int, short>> tree[100000];
bitset<100000> removed;
int current_centroid, BIT[100000];
```

```
// Return subtree size internally
// and place the discovered centroid in current_centroid
int find_centroid(const int n, const int u, const int p
    = -1) {
  if (n == 1) { current_centroid = u; return 0; }
  int subtree_sum = 0;
  for (const auto &[v, w] : tree[u]) if (v != p && !
      removed[v]) {
      subtree_sum += find_centroid(n, v, u);
      if (current_centroid > -1) return 0;
      if (subtree_sum >= n >> 1) { current_centroid = u;
          return 0; }
    }
  return subtree_sum + 1;
}
void DFS(const int u, const int p, const int length,
    const int weight, vector<pair<int, int>> &record) {
  record.emplace_back(weight, length);
  for (const auto &[v, w] : tree[u]) if (v != p && !
      removed[v])
      DFS(v, u, length + 1, weight + w, record);
}
bool greater_size(const vector<pair<int, int>> &v, const
    vector<pair<int, int>> &w) {
  return v.size() > w.size();
}
long long centroid_decomposition(const int n, int u) {
  long long ans = 0;
  // Step 1: find the centroid
  current_centroid = -1; find_centroid(n, u);
  removed[u = current_centroid] = true;
  // Step 2: DFS from the centroid (again)
  // and continue the centroid decomposition
  vector<vector<pair<int, int>>> root2subtree_paths;
  for (const auto &[v, w] : tree[u]) if (!removed[v]) {
      root2subtree_paths.emplace_back();
      DFS(v, u, 1, w, root2subtree_paths.back());
      // Sort mainly according to weight
      ranges::sort(root2subtree_paths.back());
      ans += centroid_decomposition(root2subtree_paths.
          back().size(), v);
    }
  for (const auto &v : root2subtree_paths)
    for (const auto &[weight, length] : v)
      if (length <= l && weight <= w) ++ans;
  // Step 3: optimal merging
  ranges::make_heap(root2subtree_paths, greater_size);
  while (root2subtree_paths.size() > 1) {
    ranges::pop_heap(root2subtree_paths, greater_size);
    // Merge front() (with maybe larger size) and back()
    // Count cross-centroid paths
    memset(BIT, 0, root2subtree_paths.back().size() *
        sizeof(int));
    auto p = root2subtree_paths.front().crbegin();
    for (auto q = root2subtree_paths.back().cbegin(); q
        != root2subtree_paths.back().cend(); ++q) {
      int L;
      while (p != root2subtree_paths.front().crend()
            && p->first + q->first > w) {
        L = min(l - p->second,
                static_cast<int>(root2subtree_paths.back
                    ().size()));
        while (L > 0) { ans += BIT[L - 1]; L -= L & -L;
            }
        ++p;
      }
      L = q->second;
      while (L <= static_cast<int>(root2subtree_paths.
          back().size()))
      {
        ++BIT[L - 1]; L += L & -L;
      }
    }
    while (p != root2subtree_paths.front().crend()) {
      int L = min(l - p++->second, static_cast<int>(
          root2subtree_paths.back().size()));
      while (L > 0) { ans += BIT[L - 1]; L -= L & -L; }
    }
    // Actually merge the lists
    vector<pair<int, int>> buffer;
    buffer.reserve(root2subtree_paths.front().size() +
        root2subtree_paths.back().size());
    ranges::merge(root2subtree_paths.front(),
        root2subtree_paths.back(), back_inserter(buffer)
        );

    root2subtree_paths.pop_back();
    ranges::pop_heap(root2subtree_paths, greater_size);
    root2subtree_paths.back() = move(buffer);
    ranges::push_heap(root2subtree_paths, greater_size);
  }
  return ans;
}
int main() {
  ios_base::sync_with_stdio(false);
  int n; cin >> n >> l >> w;
  for (int i = 1; i < n; ++i) {
    int p; short w; cin >> p >> w;
    tree[--p].emplace_back(i, w);
    tree[i].emplace_back(p, w);
  }
  cout << centroid_decomposition(n, 0) << endl;
}
```

5.6 Disjoint-Set

```
#include <bits/stdc++.h>
using namespace std;

struct DisjointSet {
  static const int maxn = (int)5e5 + 5;
  int n, fa[maxn], sz[maxn];
  vector<pair<int*, int>> h;
  vector<int> sp;

  void init(int _n) {
    n = _n;
    for (int i = 0 ; i < n ; ++i) {
      fa[i] = i;
      sz[i] = 1;
    }
    sp.clear(); h.clear();
  }
  void assign(int *k, int v) {
    h.push_back({k, *k});
    *k = v;
  }
  void save() { sp.push_back((int)h.size()); }
  void undo() {
    assert(!sp.empty());
    int last = sp.back(), cnt = 0; sp.pop_back();
    while (h.size() > last) {
      auto x = h.back(); h.pop_back();
      *x.first = x.second;
      cnt++;
    }
    n += cnt / 2;
  }
  int f(int x) {
    while (fa[x] != x) x = fa[x];
    return x;
  }
  bool merge(int x, int y) {
    x = f(x); y = f(y);
    if (x == y) return 0;
    if (sz[x] < sz[y]) swap(x, y);
    assign(&sz[x], sz[x] + sz[y]);
    assign(&fa[y], x);
    n--;
    return 1;
  }
} djs;
```

5.7 Heavy-Light-Decomposition

```
#include <bits/stdc++.h>
using namespace std;
const int N = 2e5 + 5;
#define eb emplace_back
int t, n, q, seg[N << 1]; // t := time-stamp
int sz[N], fa[N], dep[N], to[N], fr[N], dfn[N], arr[N];
// size, father, depth, to-heavy-child, from-head, dfs-
    order, a_i value
vector<int> g[N];
```

```
void upd(int x, int v) {
  for (seg[x += n] = v; x > 1; x >>= 1)
    seg[x >> 1] = max(seg[x], seg[x ^ 1]);
}
int qry(int l, int r) { // [l, r]
  int ret = -1e9; // -max
  for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
    if (l & 1) ret = max(ret, seg[l++]);
    if (r & 1) ret = max(ret, seg[--r]);
  }
  return ret;
}
void dfs(int x, int p) {
  sz[x] = 1, fa[x] = p, to[x] = -1, dep[x] = ~p ? dep[p]
      + 1 : 0;
  for (auto i : g[x])
    if (i != p) {
      dfs(i, x);
      if (to[x] == -1 || sz[i] > sz[to[x]]) to[x] = i;
      sz[x] += sz[i];
    }
}
void dfs2(int x, int f) {
  fr[x] = f, dfn[x] = ++t, upd(dfn[x], arr[x]);
  if (to[x] != -1) dfs2(to[x], f);
  for (auto i : g[x])
    if (i != fa[x] && i != to[x]) dfs2(i, i);
}
int qry2(int u, int v) { // query on tree
  int fu = fr[u], fv = fr[v], ret = -1e9;
  while (fu != fv) {
    if (dep[fu] < dep[fv]) swap(fu, fv), swap(u, v);
    ret = max(ret, qry(dfn[fu], dfn[u])); // interval: [
        dfn[fu], dfn[u]]
    u = fa[fu], fu = fr[u];
  }
  if (dep[u] > dep[v]) swap(u, v);
  // u is the LCA
  ret = max(ret, qry(dfn[u], dfn[v]));
  return ret;
}
int main() {
  ios::sync_with_stdio(false), cin.tie(nullptr);
  cin >> n >> q;
  for (int i = 1; i <= n; i++) cin >> arr[i];
  for (int i = 1, a, b; i < n; i++)
    cin >> a >> b, g[a].eb(b), g[b].eb(a);
  dfs(1, -1), dfs2(1, 1);
  while (q--) {
    int op; cin >> op;
    if (op == 1) {
      int x, v; cin >> x >> v, arr[x] = v, upd(dfn[x], v
          );
    }
    else {
      int a, b; cin >> a >> b;
      cout << qry2(a, b) << '\n';
    }
  }
}
```

5.8 KSP

```
#include <bits/stdc++.h>
#define debug(a) cout << #a << " = " << a << "\n";
using namespace std;
using ll = long long;
ll MOD = 1000000007LL;

int n, m, k;
vector<pair<int, ll>> v[100009], r[100009];
int cnt[100009];

int main() {
  ios::sync_with_stdio(0); cin.tie(0);

  cin >> n >> m >> k;
  for (int i = 1; i <= m; ++ i) {
    int a, b; ll c;
    cin >> a >> b >> c;
    v[a].push_back({b, c});
    r[b].push_back({a, c});
  }

  vector<ll> d(n + 1, MOD * MOD);
  d[n] = 0;
  priority_queue<pair<ll, int>, vector<pair<ll, int>>,
      greater<pair<ll, int>>> pq;
  pq.push({d[n], n});

  while (!pq.empty()) {
    auto [t, p] = pq.top();
    pq.pop();
    if (d[p] != t) continue;
    for (auto [i, c] : r[p]) if (d[p] + c < d[i]) {
        d[i] = d[p] + c;
        pq.push({d[i], i});
      }
  }

  priority_queue<tuple<ll, ll, int>, vector<tuple<ll, ll
      , int>>, greater<tuple<ll, ll, int>>> pq2;
  pq2.push({d[1], 0, 1});
  vector<ll> ans;

  while (!pq2.empty() && (int)ans.size() < k) {
    auto [t, f, p] = pq2.top();
    pq2.pop();
    if (cnt[p] == k) continue;
    cnt[p] += 1;

    if (p == n) ans.push_back(t);
    for (auto [i, c] : v[p]) {
      pq2.push({f + c + d[i], f + c, i});
    }
  }

  for (auto i : ans) cout << i << " ";
  cout << "\n";
}
```

5.9 LCA

```
#define eb emplace_back
const int N = 2e5 + 5, logN = __lg(N) + 1, inf = 1e9;
int n, q, logn;
int dep[N], fa[N][logN];
vector<int> g[N];

void dfs(int x, int p) {
  dep[x] = ~p ? dep[p] + 1 : 0;
  fa[x][0] = p;
  for (int i = 1; (1 << i) <= dep[x]; i++)
    fa[x][i] = fa[fa[x][i - 1]][i - 1];
  for (const auto& u : g[x])
    if (u != p) dfs(u, x);
}

int LCA(int u, int v) {
  if (dep[u] > dep[v]) swap(u, v);
  for (int i = 0; i < logn; i++)
    if ((dep[v] - dep[u]) >> i & 1) v = fa[v][i];
  if (u == v) return u;
  for (int i = logn - 1; i >= 0; i--)
    if (fa[u][i] != fa[v][i])
      u = fa[u][i], v = fa[v][i];
  return fa[u][0];
}
// g[a].eb(b)
// dfs(root, -1)
// query -> LCA(u, v)
// distance of (u, v) = dep[u] + dep[v] - 2 * dep[LCA(u,
    v)]
```

5.10 SCC(Kosaraju)

```
#define eb emplace_back
const int N = 2e5 + 5;
vector<int> g[N], rg[N], ord;
int scc[N];
bool v[N];
void rdfs(int x) {
  v[x] = 1;
  for (const auto& i : rg[x])
```

```
    if (!v[i]) rdfs(i);
  ord.eb(x);
}
void dfs(int x, int nscc) {
  scc[x] = nscc;
  for (const auto& i : g[x])
    if (scc[i] == -1) dfs(i, nscc);
}
void kosaraju(int n) {
  memset(v, 0, sizeof(v));
  memset(scc, -1, sizeof(scc));
  for (int i = 0; i < n; i++)
    if (!v[i]) rdfs(i);
  int nscc = 0;
  for (int i = n - 1; i >= 0; i--) {
    int x = ord[i];
    if (scc[x] == -1)
      dfs(x, nscc++);
  }
}
```

5.11 SCC(Tarjan)

```
#define ep emplace
const int N = 2e5 + 5;
int d[N], low[N], scc[N], ins[N], nscc;
vector<int> g[N];
stack<int> st;
void dfs(int x, int p) {
  d[x] = ~p ? d[p] + 1 : 1, low[x] = d[x];
  st.ep(x), ins[x] = 1;
  for (const auto& i : g[x]) {
    if (!d[i]) dfs(i, x), low[x] = min(low[x], low[i]);
    else if (ins[i]) low[x] = min(low[x], d[i]);
  }
  if (d[x] == low[x]) {
    nscc++;
    int tmp;
    do tmp = st.top(), st.pop(), scc[tmp] = nscc, ins[
        tmp] = 0; while (tmp != x);
  }
}
```

5.12 Tree-Centroid

```
const int N = 2e5 + 5, inf = 1e9;

vector<int> g[N];
int n, sz[N], center, csize; // set csize = inf

void dfs(int x, int p) {
  int mxsub = 0;
  sz[x] = 1;
  for (const auto& i : g[x])
    if (i != p) dfs(i, x), sz[x] += sz[i], mxsub = max(
        mxsub, sz[i]);
  mxsub = max(mxsub, n - sz[x]);
  if (mxsub < csize) center = x, csize = mxsub;
}
```

6 Math

6.1 Big-Int

```
#include <bits/stdc++.h>
using namespace std;

template<typename T>
inline string to_string(const T& x) {
  stringstream ss;
  return ss << x, ss.str();
}
using ll = long long;
struct bigN: vector<ll> {
  const static int base = 1000000000, width = log10(base
      );
  bool negative;
  bigN(const_iterator a, const_iterator b): vector<ll>(a
      , b) {}
  bigN(string s) {
    if (s.empty()) return;
    if (s[0] == '-')negative = 1, s = s.substr(1);
    else negative = 0;
    for (int i = int(s.size()) - 1; i >= 0; i -= width)
        {
      ll t = 0;
      for (int j = max(0, i - width + 1); j <= i; ++j)
        t = t * 10 + s[j] - '0';
      push_back(t);
    }
    trim();
  }
  template<typename T>
  bigN(const T &x): bigN(to_string(x)) {}
  bigN(): negative(0) {}
  void trim() {
    while (size() && !back())pop_back();
    if (empty()) negative = 0;
  }
  void carry(int _base = base) {
    for (size_t i = 0; i < size(); ++i) {
      if (at(i) >= 0 && at(i) < _base) continue;
      if (i + 1u == size())push_back(0);
      int r = at(i) % _base;
      if (r < 0)r += _base;
      at(i + 1) += (at(i) - r) / _base;
      at(i) = r;
    }
  }
  int abscmp(const bigN &b) const {
    if (size() > b.size()) return 1;
    if (size() < b.size()) return -1;
    for (int i = int(size()) - 1; i >= 0; --i) {
      if (at(i) > b[i]) return 1;
      if (at(i) < b[i]) return -1;
    }
    return 0;
  }
  int cmp(const bigN &b) const {
    if (negative != b.negative) return negative ? -1 :
        1;
    return negative ? -abscmp(b) : abscmp(b);
  }
  bool operator<(const bigN&b) const {return cmp(b) <
      0;}
  bool operator>(const bigN&b) const {return cmp(b) >
      0;}
  bool operator<=(const bigN&b) const {return cmp(b) <=
      0;}
  bool operator>=(const bigN&b) const {return cmp(b) >=
      0;}
  bool operator==(const bigN&b) const {return !cmp(b);}
  bool operator!=(const bigN&b) const {return cmp(b) !=
      0;}
  bigN abs() const {
    bigN res = *this;
    return res.negative = 0, res;
  }
  bigN operator-() const {
    bigN res = *this;
    return res.negative = !negative, res.trim(), res;
  }
  bigN operator+(const bigN &b) const {
    if (negative) return -(-(*this) + (-b));
    if (b.negative) return *this - (-b);
    bigN res = *this;
    if (b.size() > size()) res.resize(b.size());
    for (size_t i = 0; i < b.size(); ++i) res[i] += b[i
        ];
    return res.carry(), res.trim(), res;
  }
  bigN operator-(const bigN &b) const {
    if (negative) return -(-(*this) - (-b));
    if (b.negative) return *this + (-b);
    if (abscmp(b) < 0) return -(b - (*this));
    bigN res = *this;
    if (b.size() > size()) res.resize(b.size());
    for (size_t i = 0; i < b.size(); ++i) res[i] -= b[i
        ];
    return res.carry(), res.trim(), res;
  }
  bigN convert_base(int old_width, int new_width) const
      {
```

```
  vector<long long> p(max(old_width, new_width) + 1,
      1);
  for (size_t i = 1; i < p.size(); ++i)p[i] = p[i - 1]
      * 10;
  bigN ans;
  long long cur = 0;
  int cur_id = 0;
  for (size_t i = 0; i < size(); ++i) {
    cur += at(i) * p[cur_id];
    cur_id += old_width;
    while (cur_id >= new_width) {
      ans.push_back(cur % p[new_width]);
      cur /= p[new_width];
      cur_id -= new_width;
    }
  }
  return ans.push_back(cur), ans.trim(), ans;
}
bigN karatsuba(const bigN &b) const {
  bigN res; res.resize(size() * 2);
  if (size() <= 32) {
    for (size_t i = 0; i < size(); ++i)
      for (size_t j = 0; j < size(); ++j)
        res[i + j] += at(i) * b[j];
    return res;
  }
  size_t k = size() / 2;
  bigN a1(begin(), begin() + k);
  bigN a2(begin() + k, end());
  bigN b1(b.begin(), b.begin() + k);
  bigN b2(b.begin() + k, b.end());
  bigN a1b1 = a1.karatsuba(b1);
  bigN a2b2 = a2.karatsuba(b2);
  for (size_t i = 0; i < k; ++i)a2[i] += a1[i];
  for (size_t i = 0; i < k; ++i)b2[i] += b1[i];
  bigN r = a2.karatsuba(b2);
  for (size_t i = 0; i < a1b1.size(); ++i)r[i] -= a1b1
      [i];
  for (size_t i = 0; i < a2b2.size(); ++i)r[i] -= a2b2
      [i];
  for (size_t i = 0; i < r.size(); ++i)res[i + k] += r
      [i];
  for (size_t i = 0; i < a1b1.size(); ++i)res[i] +=
      a1b1[i];
  for (size_t i = 0; i < a2b2.size(); ++i)res[i + size
      ()] += a2b2[i];
  return res;
}
bigN operator*(const bigN &b) const {
  const static int mul_base = 1000000, mul_width =
      log10(mul_base);
  bigN A = convert_base(width, mul_width);
  bigN B = b.convert_base(width, mul_width);
  int n = max(A.size(), B.size());
  while (n & (n - 1))++n;
  A.resize(n), B.resize(n);
  bigN res = A.karatsuba(B);
  res.negative = negative != b.negative;
  res.carry(mul_base);
  res = res.convert_base(mul_width, width);
  return res.trim(), res;
}
bigN operator*(long long b) const {
  bigN res = *this;
  if (b < 0)res.negative = !negative, b = -b;
  for (size_t i = 0, is = 0; i < res.size() || is; ++i
      ) {
    if (i == res.size()) res.push_back(0);
    long long a = res[i] * b + is;
    is = a / base;
    res[i] = a % base;
  }
  return res.trim(), res;
}
bigN operator/(const bigN &b) const {
  int norm = base / (b.back() + 1);
  bigN x = abs() * norm;
  bigN y = b.abs() * norm;
  bigN q, r;
  q.resize(x.size());
  for (int i = int(x.size()) - 1; i >= 0; --i) {
    r = r * base + x[i];
    int s1 = r.size() <= y.size() ? 0 : r[y.size()];
    int s2 = r.size() < y.size() ? 0 : r[y.size() -
        1];
    int d = (ll(base) * s1 + s2) / y.back();
    r = r - y * d;
    while (r.negative) r = r + y, --d;
    q[i] = d;
  }
  q.negative = negative != b.negative;
  return q.trim(), q;
}
bigN operator%(const bigN &b) const {
  return *this - (*this / b) * b;
}
friend istream& operator>>(istream &ss, bigN &b) {
  string s;
  return ss >> s, b = s, ss;
}
friend ostream& operator<<(ostream &ss, const bigN &b)
    {
  if (b.negative) ss << '-';
  ss << (b.empty() ? 0 : b.back());
  for (int i = int(b.size()) - 2; i >= 0; --i)
    ss << setw(width) << setfill('0') << b[i];
  return ss;
}
template<typename T>
operator T() {
  stringstream ss;
  ss << *this;
  T res;
  return ss >> res, res;
}
};
```

6.2 Extgcd

```
pair<ll, ll> extgcd(ll a, ll b) {
  if (b == 0) return {1, 0};
  auto [xp, yp] = extgcd(b, a % b);
  return {yp, xp - a / b * yp};
}
```

6.3 Karatsuba

```
void karatsuba(const vector<ll>& f, const vector<ll>& g,
    vector<ll>& c, int n) {
  if (n <= 32) {
    for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
        c[i + j] += f[i] * g[j];
    return;
  }
  vector<ll> f1(n / 2), f2(n / 2), g1(n / 2), g2(n / 2);
  copy(f.begin(), f.begin() + n / 2, f1.begin()), copy(f
      .begin() + n / 2, f.end(), f2.begin());
  copy(g.begin(), g.begin() + n / 2, g1.begin()), copy(g
      .begin() + n / 2, g.end(), g2.begin());
  vector<ll> t1(n), t2(n), t3(n);
  karatsuba(f1, g1, t1, n / 2), karatsuba(f2, g2, t2, n
      / 2);
  for (int i = 0; i < n / 2; i++) f1[i] += f2[i];
  for (int i = 0; i < n / 2; i++) g1[i] += g2[i];
  karatsuba(f1, g1, t3, n / 2);
  for (int i = 0; i < n; i++) t3[i] -= t1[i] + t2[i];
  for (int i = 0; i < n; i++)
    c[i] += t1[i], c[i + n] += t2[i], c[i + n / 2] += t3
        [i];
}
void mul(const vector<ll>& a, const vector<ll>& b,
    vector<ll>& c) {
  int n = a.size(), m = b.size(), t = max(n, m), p = 1;
  while (p < t) p <<= 1;
  vector<ll> aa(p), bb(p);
  copy(a.begin(), a.end(), aa.begin()), copy(b.begin(),
      b.end(), bb.begin());
  c.assign(p << 1, 0), karatsuba(aa, bb, c, p), c.resize
      (n + m - 1);
}
```

6.4 Linear-Sieve

```
vector<bool> isp;
vector<int> p;
void sieve(int n) {
  p.clear(), isp.assign(n + 1, 1);
  isp[0] = isp[1] = 0;
  for (int i = 2; i <= n; i++) {
    if (isp[i]) p.eb(i);
    for (const auto& x : p) {
      if (1LL * i * x > n) break;
      isp[i * x] = 0;
      if (i % x == 0) break;
    }
  }
}
```

6.5 Matrix

```
template <typename T> using vec = vector<T>;
template <typename T> using matrix = vec<vec<T>>;
constexpr int mod = 1e9 + 7;
template <typename T>
matrix<T> operator*(const matrix<T>& a, const matrix<T>&
    b) {
  int n = a.size(), r = b.size(), m = b.front().size();
  matrix<T> ret(n, vec<T>(m));
  for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
      for (int k = 0; k < r; k++)
        ret[i][j] += 1LL * a[i][k] * b[k][j] % mod, ret[
            i][j] %= mod;
  return ret;
}
```

6.6 Miller-Rabin

```
using ll = ll;
ll mod_mul(ll a, ll b, ll m) {
  a %= m, b %= m;
  ll y = (ll)((double)a * b / m + 0.5); /* fast for m <
      2^58 */
  ll r = (a * b - y * m) % m;
  return r < 0 ? r + m : r;
}
template<typename T>
T pow(T a, T b, T mod) { //a^b%mod
  T ans = 1;
  for (; b; a = mod_mul(a, a, mod), b >>= 1)
    if (b & 1)ans = mod_mul(ans, a, mod);
  return ans;
}
int sprp[3] = {2, 7, 61}; // range of int
int llsprp[7] = {2, 325, 9375, 28178, 450775, 9780504,
    1795265022}; // range of unsigned ll
template<typename T>
bool isprime(T n, int *sprp, int num) {
  if (n == 2)return 1;
  if (n < 2 || n % 2 == 0) return 0;
  int t = 0;
  T u = n - 1;
  for (; u % 2 == 0; ++t)u >>= 1;
  for (int i = 0; i < num; ++i) {
    T a = sprp[i] % n;
    if (a == 0 || a == 1 || a == n - 1) continue;
    T x = pow(a, u, n);
    if (x == 1 || x == n - 1) continue;
    for (int j = 1; j < t; ++j) {
      x = mod_mul(x, x, n);
      if (x == 1) return 0;
      if (x == n - 1) break;
    }
    if (x == n - 1) continue;
    return 0;
  }
  return 1;
}
```

6.7 NTT

```
const int G = 3, P = 998244353;
const int sval = 100, split = log10(sval);
int fpow(int x, int y) {
  int ret = 1;
  for (; y; y >>= 1, x = 1LL * x * x % P)
    if (y & 1) ret = 1LL * ret * x % P;
  return ret;
}
void ntt(vector<int>& x, int lim, int opt) {
  for (int i = 1, j = 0; i < lim; i++) {
    for (int k = lim >> 1; !((j ^= k) & k); k >>= 1);
    if (i < j) swap(x[i], x[j]);
  }
  for (int m = 2; m <= lim; m <<= 1) {
    int k = m >> 1;
    int gn = fpow(G, (P - 1) / m);
    for (int i = 0; i < lim; i += m) {
      int g = 1;
      for (int j = 0; j < k; ++j, g = 1LL * g * gn % P)
          {
        int tmp = 1LL * x[i + j + k] * g % P;
        x[i + j + k] = (x[i + j] - tmp + P) % P;
        x[i + j] = (x[i + j] + tmp) % P;
      }
    }
  }
  if (opt == -1) {
    reverse(x.begin() + 1, x.begin() + lim);
    int inv = fpow(lim, P - 2);
    for (int i = 0; i < lim; ++i)
      x[i] = 1LL * x[i] * inv % P;
  }
}
vector<int> mul(vector<int> a, vector<int> b) {
  int lim = 1, n = a.size(), m = b.size();
  while (lim < (n + m - 1)) lim <<= 1;
  a.resize(lim + 1), b.resize(lim + 1);
  ntt(a, lim, 1), ntt(b, lim, 1);
  for (int i = 0; i < lim; ++i)
    a[i] = 1LL * a[i] * b[i] % P;
  ntt(a, lim, -1);
  int len = 0;
  for (int i = 0; i < lim; ++i) {
    if (a[i] >= sval) len = i + 1, a[i + 1] += a[i] /
        sval, a[i] %= sval;
    if (a[i]) len = max(len, i);
  }
  while (a[len] >= sval) a[len + 1] += a[len] / sval, a[
      len] %= sval, len++;
  return a.resize(len + 1), a;
}
void print(const vector<int>& v) {
  if (!v.size()) return;
  cout << v.back();
  for (int i = v.size() - 2; ~i; --i)
    cout << setfill('0') << setw(split) << v[i];
  cout << '\n';
}
int main() {
  ios::sync_with_stdio(false), cin.tie(nullptr);
  string stra, strb;
  while (cin >> stra >> strb) {
    vector<int> a((stra.size() + split - 1) / split);
    vector<int> b((strb.size() + split - 1) / split);
    int tmp = stra.size();
    for (auto& i : a)
      tmp -= split, i = atoi(stra.substr(max(0, tmp),
          min(split, split + tmp)).data());
    tmp = strb.size();
    for (auto& i : b)
      tmp -= split, i = atoi(strb.substr(max(0, tmp),
          min(split, split + tmp)).data());
    print(mul(a, b));
  }
  return 0;
}
```

6.8 Pollard-Rho

```
// does not work when n is prime
ll add(ll a, ll b, ll m) {return (a += b) > m ? a - m :
    a;}
ll mul(ll a, ll b, ll m) {
  a %= m, b %= m;
  ll y = (ll)((double)a * b / m + 0.5); /* fast for m <
      2^58 */
```

```
  ll r = (a * b - y * m) % m;
  return r < 0 ? r + m : r;
}
ll f(ll x, ll mod) { return add(mul(x, x, mod), 1, mod);
    }
ll pollard_rho(ll n) {
  if (!(n & 1)) return 2;
  while (true) {
    ll y = 2, x = rand() % (n - 1) + 1, res = 1, tmp =
        1;
    for (int sz = 2; res == 1; sz *= 2, y = x) {
      for (int i = 0, t = 0; i < sz && res <= 1; i++, t
          ++) {
        x = f(x, n); tmp = mul(tmp, abs(x - y), n);
        if (!(t & 31) || i + 1 == sz) res = __gcd(tmp, n
            ), tmp = 1;
      }
    }
    if (res != 0 && res != n) return res;
  }
}
```

6.9 Theorem

- Chinese Remainder Theorem:
  if $m_1, m_2, \ldots, m_n$ are coprime with each other, then
  $$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \ldots \\ x \equiv a_n \pmod{m_n} \end{cases}$$
  could be solved by Chinese Remainder Theorem.
  1. Let $M = m_1 \times m_2 \times \cdots \times m_n$ and let $M_i = \frac{M}{m_i}$
  2. Find t_i s.t. $M_i \cdot t_i \equiv 1 \pmod{m_i}$ by Extended Euclidean algorithm.
  3. For any integer k, there exists general solution
  $x = a_1 t_1 M_1 + a_2 t_2 M_2 + \cdots + a_n t_n M_n + kM = kM + \sum_{i=1}^{n} a_i t_i M_i$

7 Others

7.1 Mo

```
#include <bits/stdc++.h>
using namespace std;

const int N = 2e5 + 5, sqN = sqrt(N) + 5;
int a[N], ans[N], n, q, sz; // maybe need blk[sqN];

struct Query {
  int ql, qr, id;
  bool operator<(const Query& b) const {
    int aa = ql / sz, bb = b.ql / sz;
    if (aa != bb) return aa < bb;
    else return qr < b.qr;
  }
} Q[N];

void add(int x) {}
void sub(int x) {}
int qry(int k) {}

int main() {
  ios::sync_with_stdio(false), cin.tie(nullptr);
  cin >> n >> q, sz = sqrt(n);
  for (int i = 0; i < n; i++) cin >> a[i];
  for (int i = 0, ql, qr; i < q; i++)
    cin >> ql >> qr, Q[i] = {ql - 1, qr - 1, i};
  // Mo's algorithm
  sort(Q, Q + q); /* remember initialize arrays */
  int l = 0, r = -1;
  for (int i = 0; i < q; i++) {
    auto [ql, qr, k, id] = Q[i];
    while (r < qr) add(a[++r]);
    while (r > qr) sub(a[r--]);
    while (l < ql) sub(a[l++]);
    while (l > ql) add(a[--l]);
    ans[id] = qry(k);
  }
  for (int i = 0; i < q; i++) cout << ans[i] << '\n';
}
```

7.2 Partial-Ordering

```
// O(n log^2 n)
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 1e5 + 5, M = 2e5 + 5;
int n, K, cnt, ans[N];
struct node {
  int x, y, z, v, ans, tag, id;
  node() { ans = tag = v = x = y = z = 0; }
  friend bool operator==(const node &a, const node &b) {
    return (a.x == b.x) && (a.y == b.y) && (a.z == b.z);
  }
} a[N], t[N];
bool cmp1(const node &a, const node &b) {
  if (a.x != b.x) return a.x < b.x;
  if (a.y != b.y) return a.y < b.y;
  return a.z < b.z;
}
bool cmp2(const node &a, const node &b) {
  if (a.y != b.y) return a.y < b.y;
  if (a.tag != b.tag) return a.tag < b.tag;
  return a.id < b.id;
}
#define lowbit(x) (x & -x)
int bit[M];
void add(int p, int x) {
  for (; p <= K; p += lowbit(p)) bit[p] += x;
}
int query(int p) {
  int ret = 0;
  for (; p; p -= lowbit(p)) ret += bit[p];
  return ret;
}
void CDQ(int l, int r) {
  if (l == r) return;
  int mid = (l + r) >> 1;
  CDQ(l, mid); CDQ(mid + 1, r);
  for (int i = l; i <= r; ++i) a[i].id = i;
  for (int i = l; i <= mid; ++i) a[i].tag = 0;
  for (int i = mid + 1; i <= r; ++i) a[i].tag = 1;
  sort(a + l, a + r + 1, cmp2);
  for (int i = l; i <= r; ++i) {
    if (!a[i].tag) add(a[i].z, a[i].v);
    else a[i].ans += query(a[i].z);
  }
  for (int i = l; i <= r; ++i)
    if (!a[i].tag) add(a[i].z, -a[i].v);
}
int main() {
  cin >> n >> K;
  for (int i = 1; i <= n; ++i) cin >> a[i].x >> a[i].y
      >> a[i].z, a[i].v = 1;
  sort(a + 1, a + n + 1, cmp1);
  cnt = 1;
  for (int i = 2; i <= n; ++i) {
    if (a[i] == a[cnt]) ++a[cnt].v;
    else a[++cnt] = a[i];
  }
  CDQ(1, cnt);
  // let ans[i] denote that the number of (aj<=ai && bj
      <=bi && cj<=ci) for i != j
  for (int i = 1; i <= cnt; ++i) ans[a[i].ans + a[i].v -
       1] += a[i].v;
  for (int i = 0; i < n; ++i) cout << ans[i] << '\n';
  return 0;
}
```

8 String

8.1 AC

```
struct ACautomata {
  struct Node {
    int cnt;
    Node *go[26], *fail, *dic;
    Node () {
      cnt = 0, fail = 0, dic = 0;
      memset(go, 0, sizeof(go));
    }
  } pool[1048576], *root;
  int nMem;
  Node* new_Node() {
    pool[nMem] = Node();
```

```
    return &pool[nMem++];
  }
  void init() { nMem = 0, root = new_Node(); }
  void add(const string &str) { insert(root, str, 0); }
  void insert(Node *cur, const string &str, int pos) {
    for (int i = pos; i < str.size(); i++) {
      if (!cur->go[str[i] - 'a'])
        cur->go[str[i] - 'a'] = new_Node();
      cur = cur->go[str[i] - 'a'];
    }
    cur->cnt++;
  }
  void make_fail() {
    queue<Node*> que;
    que.push(root);
    while (!que.empty()) {
      Node* fr = que.front(); que.pop();
      for (int i = 0; i < 26; i++) {
        if (fr->go[i]) {
          Node *ptr = fr->fail;
          while (ptr && !ptr->go[i]) ptr = ptr->fail;
          fr->go[i]->fail = ptr = (ptr ? ptr->go[i] :
              root);
          fr->go[i]->dic = (ptr->cnt ? ptr : ptr->dic);
          que.push(fr->go[i]);
        }
      }
    }
  }
} AC;
```

8.2 Hash

```
struct Hash {
  vector<ll> h;
  vector<ll> f;
  Hash(string s, int p = 127) {
    h.assign((int)s.size() + 1, 0);
    f.assign((int)s.size() + 1, 0);
    f[0] = 1;
    for (int i = 1; i <= (int)s.size(); ++ i) f[i] = f[i
        - 1] * p % MOD;
    for (int i = 1; i <= (int)s.size(); ++ i)
      h[i] = (h[i - 1] * p + s[i - 1]) % MOD;
  }
  int query(int l, int r) { // 0-based [l, r]
    if (r < l) return 0;
    return ((h[r + 1] - h[l] * f[r - l + 1]) % MOD + MOD
        ) % MOD;
  }
};
```

8.3 KMP

```
#define pb push_back
const int N = 1e6 + 5;
int F[N];
vector<int> match(string A, string B) {
  vector<int> ans;
  F[0] = -1, F[1] = 0;
  for (int i = 1, j = 0; i < (int)B.size(); F[++i] = ++j
      ) {
    if (B[i] == B[j]) F[i] = F[j]; // optimize
    while (j != -1 && B[i] != B[j]) j = F[j];
  }
  for (int i = 0, j = 0; i < (int)A.size(); ++i) {
    while (j != -1 && A[i] != B[j]) j = F[j];
    if (++j == (int)B.size()) ans.pb(i + 1 - j), j = F[j
        ];
  }
  return ans;
}
```

8.4 Manacher

```
// P[2i] := max 2j+1: s[i-j, i+j] is palindromic
// P[2i-1] := max 2j: s[i-j, i+j) is palindromic
// maximal palindrome: s.substr((1 + i - P[i]) >> 1, P[i
    ])
vector<unsigned> Manacher(const string &s) {
  unsigned L = 0, R = 1;
  vector<unsigned> P; P.reserve((s.size() << 1) - 1);
  P.push_back(1);
  for (unsigned i = 1; i < s.size(); ++i)
    for (int j = 0; j < 2; ++j) {
      if (i < R) {
        const int k = ((L + R - i) << 1) - j - 1;
        if (P[k] >> 1 < R - i - j) { P.push_back(P[k]);
            continue; }
        L = (i << 1) - R + j;
      }
      else R = (L = i) + j;
      while (L > 0 && R < s.size() && s[L - 1] == s[R])
          {--L; ++R;}
      P.push_back(R - L);
    }
  return P;
}
```

8.5 SA

```
const int N = 2e5 + 5;

string s;
int sa[N], tmp[2][N], c[N], rk[N], h[N];
// lcp(sa[i], sa[j]) = min{h[k]} where i <= k <= j

void suffix_array() {
  int *x = tmp[0], *y = tmp[1], m = 256, n = s.size();
  fill(c, c + m, 0);
  for (int i = 0; i < n; i++) c[x[i] = s[i]]++;
  partial_sum(c, c + m, c);
  for (int i = n - 1; i >= 0; i--) sa[--c[x[i]]] = i;
  for (int k = 1; k < n; k <<= 1) {
    fill(c, c + m, 0);
    for (int i = 0; i < n; i++) c[x[i]]++;
    partial_sum(c, c + m, c);
    int p = 0;
    for (int i = n - k; i < n; i++) y[p++] = i;
    for (int i = 0; i < n; i++)
      if (sa[i] >= k) y[p++] = sa[i] - k;
    for (int i = n - 1; i >= 0; i--) sa[--c[x[y[i]]]] =
        y[i];
    y[sa[0]] = p = 0;
    for (int i = 1; i < n; i++) {
      int a = sa[i], b = sa[i - 1];
      if (x[a] != x[b] || a + k >= n || b + k >= n || x[
          a + k] != x[b + k]) p++;
      y[sa[i]] = p;
    }
    if (n == p + 1) break;
    swap(x, y), m = p + 1;
  }
}

void LCP() {
  int n = s.size(), val = 0;
  for (int i = 0; i < n; i++) rk[sa[i]] = i;
  for (int i = 0; i < n; i++) {
    if (rk[i] == 0) h[rk[i]] = 0;
    else {
      if (val) val--;
      int p = sa[rk[i] - 1];
      while (val + i < n && val + p < n && s[val + i] ==
          s[val + p]) val++;
      h[rk[i]] = val;
    }
  }
}
// cin >> s, suffix_array(), LCP();
```

8.6 SAIS

```
const int N = 300010;
struct SA {
#define REP(i,n) for(int i=0;i<int(n);i++)
#define REP1(i,a,b) for(int i=(a);i<=int(b);i++)
  bool _t[N * 2]; int _s[N * 2], _sa[N * 2];
  int _c[N * 2], x[N], _p[N], _q[N * 2], hei[N], r[N];
  int operator []int i) { return _sa[i]; }
  void build(int *s, int n, int m) {
    memcpy(_s, s, sizeof(int)*n);
    sais(_s, _sa, _p, _q, _t, _c, n, m); mkhei(n);
  }
```

```
  void mkhei(int n) {
    REP(i, n) r[_sa[i]] = i;
    hei[0] = 0;
    REP(i, n) if (r[i]) {
      int ans = i > 0 ? max(hei[r[i - 1]] - 1, 0) : 0;
      while (_s[i + ans] == _s[_sa[r[i] - 1] + ans]) ans
          ++;
      hei[r[i]] = ans;
    }
  }
  void sais(int *s, int *sa, int *p, int *q, bool *t,
      int *c, int n, int z) {
    bool uniq = t[n - 1] = true, neq;
    int nn = 0, nmxz = -1, *nsa = sa + n, *ns = s + n,
        lst = -1;
#define MS0(x,n) memset((x),0,n*sizeof(*(x)))
#define MAGIC(XD) MS0(sa,n);\
memcpy(x,c,sizeof(int)*z); XD;\
memcpy(x+1,c,sizeof(int)*(z-1));\
REP(i,n) if(sa[i]&&!t[sa[i]-1]) sa[x[s[sa[i]-1]]++]=sa[i
    ]-1;\
memcpy(x,c,sizeof(int)*z);\
for(int i=n-1;i>=0;i--) if(sa[i]&&t[sa[i]-1]) sa[--x[s[
    sa[i]-1]]]=sa[i]-1;
    MS0(c, z); REP(i, n) uniq &= ++c[s[i]] < 2;
    REP(i, z - 1) c[i + 1] += c[i];
    if (uniq) { REP(i, n) sa[--c[s[i]]] = i; return; }
    for (int i = n - 2; i >= 0; i--)
      t[i] = (s[i] == s[i + 1] ? t[i + 1] : s[i] < s[i +
          1]);
    MAGIC(REP1(i, 1, n - 1) if (t[i] && !t[i - 1]) sa[--
        x[s[i]]] = p[q[i] = nn++] = i);
    REP(i, n) if (sa[i] && t[sa[i]] && !t[sa[i] - 1]) {
      neq = lst < 0 || memcmp(s + sa[i], s + lst, (p[q[
          sa[i]] + 1] - sa[i]) * sizeof(int));
      ns[q[lst = sa[i]]] = nmxz += neq;
    }
    sais(ns, nsa, p + nn, q + n, t + n, c + z, nn, nmxz
        + 1);
    MAGIC(for (int i = nn - 1; i >= 0; i--) sa[--x[s[p[
        nsa[i]]]]] = p[nsa[i]]);
  }
} sa;
int H[N], SA[N], RA[N];
void suffix_array(int* ip, int len) {
  // should padding a zero in the back
  // ip is int array, len is array length
  // ip[0..n-1] != 0, and ip[len]=0
  ip[len++] = 0; sa.build(ip, len, 128);
  memcpy(H, sa.hei + 1, len << 2); memcpy(SA, sa._sa +
      1, len << 2);
  for (int i = 0; i < len; i++) RA[i] = sa.r[i] - 1;
  // resulting height, sa array \in [0,len)
}
```

8.7 Z

```
void z_value(const char *s, int len, int *z) {
  z[0] = len;
  for (int i = 1, l = 0, r = 0; i < len; i++) {
    z[i] = i < r ? (i - l + z[i - l] < z[l] ? z[i - l] :
        r - i) : 0;
    while (i + z[i] < len && s[i + z[i]] == s[z[i]]) ++z
        [i];
    if (i + z[i] > r) l = i, r = i + z[i];
  }
}
```